



函数

▼ 函数类型

在 C/C++ 中，变量是可以储存值、可以被赋值、具有地址空间的对象。

而函数不能被赋值、不能被复制、不能作为参数传递、不能做 `sizeof`、不能作为类成员（除非用函数指针或成员函数指针），但是函数依然具有自己的地址。

函数名并不代表函数地址，而是代表了函数实体，就像普通变量的名字代表变量实体而不是地址。负责存储函数地址的应该是函数指针。

函数实体既不属于变量，也并不是指针，其属于“函数类型”。

▼ 函数指针

在函数类型前加&就代表了函数地址，而函数地址的数据类型就为函数指针。

定义函数指针的方法为：

```
T (*p)(); //正确，定义了一个指向函数地址的指针
T *p();  //错误，这是定义了一个返回值为指针类型的函数
```

C++

在大多数使用场景中，函数地址（函数指针）和函数名（函数实体）可以混用。

之所以可以将函数实体和函数指针混用，是因为两者在多数情况都存在隐式转换。

但这并不代表着函数实体和函数指针的数据类型相同。

```
bool cmp(int x,int y){
    return x>y;
}
//其函数类型为bool(int,int)
//其函数指针类型为bool(*)(int,int)
```

C++

```
decltype(cmp) //判断函数本体数据类型
```

需要注意的是，函数类型不能作为参数传递，当使用函数类型作为参数时，会自动退化为函数指针。

▼ 函数类型作为函数参数

C

```
void call(void a(int, int)) {  
    a(1, 2);  
}  
/* 等价于  
void call(void (*a)(int, int)) {  
    a(1, 2);  
}*/  
// 二者本质相同，同时定义时不算作重载，而是重定义
```

同理，由于只能使用函数指针作为参数，所以传递参数时应该使用的是函数地址，不过直接使用函数实体作为参数，也会自动转化为函数地址。

▼ 函数类型作为调用参数

C++

```
call(testA);    // ✓ 推荐写法，简洁清晰  
call(&testA);   // ✓ 合法写法，但不必要
```

除此之外，函数指针只是函数地址，无法直接被调用，地址上的函数实体才是被调用的对象，但实际使用时依然可以直接通过函数指针调用函数，这也是因为自动转换。

C++

```
func(value);    // ✓ 推荐写法，简洁清晰  
(*func)(value); // ✓ 合法写法，但不必要
```

函数指针在使用 `using` 或者 `typedef` 定义类型别名时，方法与普通类型不太一样：

C++

```
typedef int (*FuncPtr)(int, int);  
using FuncPtr = int (*)(int, int);
```

两种方式等效，为 `int (*)(int, int)` 类型的函数指针定义了 `FuncPtr` 别名。

▼ 函数本质

在 C++ 中，一个函数本质上是一段具有名称、签名和地址的指令序列，能够被调用并执行特定逻辑。

特性	含义
名称	编译器记录函数标识
入口地址	指向其指令的地址，可以通过函数指针调用
调用约定	调用者和被调用者如何传参、返回、维护栈结构等规则
调用接口（签名）	参数个数、类型、返回类型

函数体本质上就是一段汇编指令序列，函数体不是数据，而是代码指令，跟变量、数组等数据区分开。

- 编译后的程序（如 `.exe` 文件）里，函数的机器码被放在代码段（通常叫 `.text` 段）；
- 代码段在程序加载到内存时，会被映射到一个连续的内存区域；
- 函数的第一条指令有一个**固定的内存地址**，称为**函数入口地址**；
- 当程序执行到调用函数的地方，CPU 会跳转（跳转指令，如 `call`）到这个地址，开始执行函数的指令序列；
- 函数执行完毕后，会通过返回指令（如 `ret`）跳回调用点继续执行。
- 函数指针就是保存函数入口地址的变量，可以通过它调用函数；

函数本身（即汇编指令的集合）不能被移动与拷贝，只能通过函数指针调用，函数地址在运行时确定，并且本次运行不会被更改。

- 编译时函数有相对地址（链接器地址）
- 程序加载到内存时，操作系统会给代码段分配一个**绝对地址**
- 程序运行时 CPU 使用的是这个绝对地址。

不过 `constexpr` 函数如果只需要在编译期调用，该函数会被编译器优化，不再生成函数体，其代表的汇编指令会直接在调用处展开，而不是具有自己的函数地址。

具有函数体的非 `constexpr` 函数则是在运行时才会被加载到内存并分配地址。

`constexpr` 函数如果也需要在运行时调用，则会在编译期调用处展开的同时生成函数体，供运行时调用。

▼ 函数编译

函数在的声明通常与定义放在不同文件，声明放在头文件，而定义实现在源文件。

这是因为头文件会被多个源文件包含，在预处理阶段之后，就相当于将头文件中的内容全部复制在源文件中。

如果将定义实现在头文件，那么在预处理之后，所有包含该头文件的编译单元都会存在这个函数的定义，这就构成了函数的重定义。

事实上，编译阶段的函数调用处，并不需要知道函数的具体定义，只需要知道**函数签名**，而**函数声明**提供了**签名信息**。

```
int add(int a, int b); // 签名是 add(int, int)
double add(double a, double b); // 签名是 add(double, double)
```

C++

在 C++ 中，**函数签名**是指函数的唯一标识，包括：

- 函数名
- 参数类型列表（不含参数名）

- 不包括返回类型 (C++ 中)
- 不包括如 `constexpr`、`inline` 等关键字

所以只要有函数声明提供的函数签名，函数调用处就不需要知道函数的定义，直到链接时才会将函数调用处的引用与函数定义的地址链接。

▼ 函数对象

函数对象本身不是函数，而是一个类或结构体的实例对象。

通过对 `()` 进行重载，就可以使实例搭配 `()` 时调用 `operator()` 函数。实例对象的 `()` 与函数的 `()` 本质是不同的，实例对象的 `()` 会调用 `operator()`，`operator()` 的 `()` 才是函数的 `()`。

比如：

```
C++
class MyAdd
{
public:
    int operator()(int v1, int v2)
    {
        return v1 + v2;
    }
};

void test01()
{
    MyAdd add;
    cout << add(10,10) << endl;
}
```

通过以上例子可以看出，`add` 本身不是函数，而是 `MyAdd` 类的一个实例，只是重载了 `()`，从而达到了类似于函数的效果。

函数对象相比于普通的函数，可以包含更多信息，因为函数对象本质为类的对象，其中可以包括其他的成员变量。

```
C++
class MyPrint
{
public:
    MyPrint()
    {
        this->count = 0;
    }
    void operator()(string test)
    {
        cout << test << endl;
        this->count++;
    }
    int count; //内部自己状态
};

int test02() {
```

```
MyPrint myPrint;
myPrint("hello world");
cout << "myPrint调用次数为：" << myPrint.count << endl;
```

▼ std::function

std::function 是一个函数包装器模板，一个 std::function 类型对象实例可以包装下列这几种可调用元素类型：函数、函数指针、类成员函数指针或任意类型的函数对象。

```
//可以表示任何以T1为返回值、T2...为参数的函数、函数指针、类成员函数指针或任意类型的函数对象
function<T1(T2...)>

//举例：
function<bool(int,int)>
```

C++

▼ 无名参数

函数参数可以只指定类型而不定义变量名，这种参数通常被称为**无名参数**。

常用于以下场景：

- 占位符：重载**后置自增/自减运算符**（如 `it++`）时，为了与前置版本（`++it`）区分，C++ 语法要求后置版本必须带有一个 `int` 参数。但这个 `int` 在函数体内部完全用不到，所以通常不写名字。
- 接口更新：原来的函数需要两个参数，但新的函数只需要一个。为了不破坏已经调用该函数的所有现有代码（保持 API 兼容），可以删掉参数名，但保留参数类型。
- 标签分发：类型萃取时经常需要传入一个空结构体作为“标签”来决定调用哪个重载函数。因为这个结构体没有数据，我们只需要它的**类型信息**来触发重载，不需要在函数里操作它。

无名参数主要针对函数的定义，而不针对函数声明。

在函数声明（.h 文件）中不写参数名是非常普遍的，函数标签本身也不依赖参数名，而只需要参数类型。函数声明中不写参数名不影响函数定义中使用参数名。

只有在函数定义（.cpp 文件）中不写参数名才称得上无名参数，因为定义中如果没有参数名，函数体内就无法使用该参数